Dynamic Method

Decide how to define a method at runtime.

```
class C
end

C.class_eval do
  define_method :my_method do
    "a dynamic method"
  end
end

obj = C.new
obj.my_method # => "a dynamic method"
```

For more information, see page 51.

Dynamic Proxy

Dynamically forward method calls to another object.

```
class MyDynamicProxy
  def initialize(target)
    @target = target
  end

  def method_missing(name, *args, &block)
    "result: #{@target.send(name, *args, &block)}"
  end
end

obj = MyDynamicProxy.new("a string")
obj.reverse # => "result: gnirts a"
```

For more information, see page 60.

Flat Scope

Use a closure to share variables between two scopes.

```
class C
  def an_attribute
    @attr
  end
end

obj = C.new
a_variable = 100
```